# HyDE for Semantic Search over Consumer Complaints: How Hypothetical Document Embeddings Work, and How They Compare with Other Semantic Search Methods

CS410 Technology Review — CCDB Search

Carson Wilkins

November 7, 2026

**Abstract**

This review studies Hypothetical Document Embeddings (HyDE) as the retrieval method for *CCDB Search*, our planned semantic search over the consumer-written complaint narratives in the CFPB Consumer Complaint Database. Part I explains how HyDE works: an LLM writes hypothetical answer documents for the query, and the mean of their embeddings and the query's embedding is used for nearest-neighbour search. Using real vectors from four BEIR collections we show how this averaged vector moves the search into the neighbourhood of the relevant documents (one SciFact claim's relevant abstract moves from rank 113 to rank 1), when it fails (unnormalized encoders, LLM output in the wrong style), and we turn these findings into design choices for CCDB Search. Part II compares HyDE with the other main families of semantic search (learned sparse SPLADE, dense bi-encoders, ColBERTv2 late interaction, hybrid RRF fusion, cross-encoder reranking and RankGPT LLM reranking) against a BM25 baseline on SciFact, NFCorpus, FiQA and TREC-COVID, after reproducing each method's published results. HyDE with bge-small is the strongest first-stage retriever on three of the four collections (+0.090 nDCG@10 over BM25 on SciFact) but costs 10–19 s per query; RankGPT is the most effective system overall, and hybrid fusion gives most of the gain for about 10 ms. Semantic search helps most where queries and documents are worded differently, which is the situation CCDB Search is built for.

## 1 Introduction

The CFPB Consumer Complaint Database (CCDB) is the U.S. Consumer Financial Protection Bureau's public collection of consumer complaints about financial products and services such as credit reporting, debt collection, mortgages, credit cards and bank accounts. Many complaints include a free-text narrative written by the consumer, published only with the consumer's consent and with personal details scrubbed. These narratives are what our project, **CCDB Search**, will retrieve.

Searching them is a vocabulary-mismatch problem. A user types a short query, for example "charged twice for an overdraft", while the complaints that match it are long first-person stories that may use none of those words. Keyword search (BM25) only finds documents that share the query's terms. *Semantic search* instead maps queries and documents to vectors so that texts with the same meaning are close even when their words differ. HyDE [7] goes one step further: it first asks an LLM to write what a matching document might look like, and searches with that.

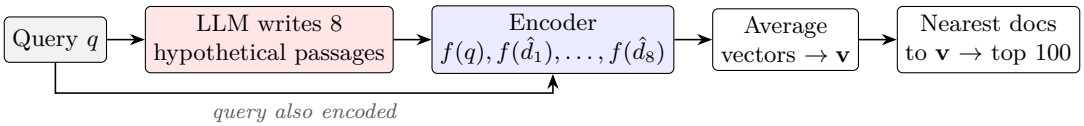
The review has two parts. **Part I** explains HyDE in depth (Section 2), measures what it does to the query vector on real data (Section 3), and derives the design of CCDB Search from those

measurements (Section 4). **Part II** compares HyDE with the other main semantic search methods on four public test collections, reproducing published results first so that the comparison can be trusted. The CCDB has no relevance judgments, so it could not be evaluated directly; the BEIR collections stand in for it.

# Part I
# HyDE for CCDB Search

## 2 How HyDE works

**Idea.** A dense retriever compares the query vector $f(q)$ with every document vector $f(d)$ and returns the nearest. But queries and documents are different kinds of text: a query is a few words ("Deleting Raptor reduces G-CSF levels."), a document is a full abstract with its own vocabulary. Their vectors live in different parts of the space, so the nearest documents to a *query* need not be the documents that *answer* it. HyDE turns retrieval into document–document similarity, which encoders are good at: an LLM *writes* $N$ plausible answer passages $\hat{d}_1, \ldots, \hat{d}_N$ ("Please write a scientific paper passage to support/refute the claim"), and the search looks for real documents that are close to those fake ones. The passages may contain invented facts; that does not matter, because they are never shown to the user, only encoded. The LLM never sees the corpus.




**The averaged vector.** The search vector is the mean of the query embedding and the $N$ passage embeddings (the paper's Eq. 8; we use $N = 8$, as the official code does):

$$\mathbf{v} \;=\; \frac{1}{N+1}\Big(f(q) + \sum_{k=1}^{N} f(\hat{d}_k)\Big).$$

Figure 1 draws this in a two-dimensional cartoon of the embedding space. Real documents form a region (gray); the short query lands outside it. Each hypothetical passage is written like a real document, so its vector lands *inside* the document region, near the documents that discuss the same topic. The passages differ in their invented details, so their vectors scatter, but they share the topic, so they scatter around the same place. Averaging cancels the sample-specific noise and keeps the shared direction; adding $f(q)$ keeps the vector anchored to what was actually asked. Search then returns the documents with the smallest angle to $\mathbf{v}$.

**Why averaging works: the score is an average of similarities.** The inner product is linear, so scoring a document against the mean vector is the same as averaging its similarity to each of the nine vectors:

$$\text{score}(d) \;=\; \mathbf{v} \cdot f(d) \;=\; \frac{1}{N+1}\Big(f(q)\cdot f(d) \;+\; \sum_{k=1}^{N} f(\hat{d}_k)\cdot f(d)\Big).$$

So HyDE is a vote: a document ranks high only if it is close to the query *and* to most of the passages. A document that matches one passage's invented detail gets one vote; a document on

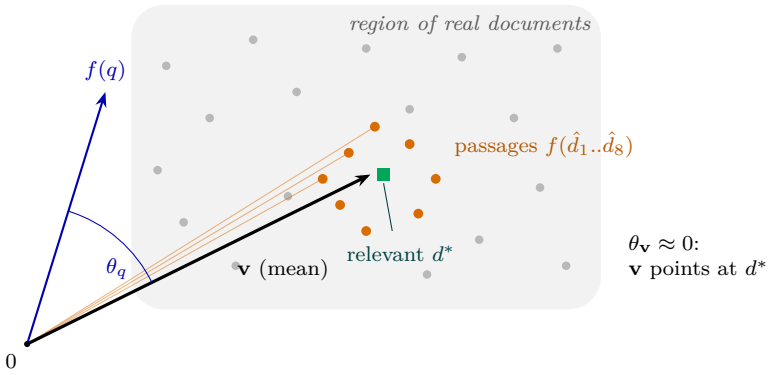
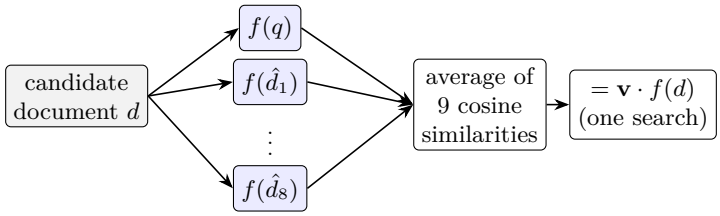



Figure 1: Cartoon of HyDE in embedding space. The query vector $f(q)$ points away from the document region; the eight hypothetical passages (orange) point into it, around the relevant document. Their mean $\mathbf{v}$ (black) has a much smaller angle to the relevant document ($\theta_{\mathbf{v}} \ll \theta_q$), so nearest-neighbour search on $\mathbf{v}$ ranks it near the top. Figure 2 shows the same picture for a real query.

the topic all passages share gets all of them. That is why hallucinated facts do little harm, and why the method still needs only one nearest-neighbour search (the mean is computed once, so the cost is the same as Dense search).




**Why vector length matters.** The mean treats vectors as arrows, so a longer arrow pulls harder. bge normalizes every vector to length 1, so the query and each passage get an equal 1/9 say. Contriever does not normalize; its vectors grow with text length. The cartoon below shows the effect we measured (Section 3): long LLM passages produce long vectors that swamp $f(q)$, and the mean drifts toward whatever the passages over-emphasize.




## 3 HyDE on real data: what the averaged vector does

To see this mechanism in the real system, we re-encoded the cached gpt-oss passages and measured, for every query with a judged-relevant document, how the query vector $f(q)$, the individual passage vectors $f(\hat{d}_k)$ and the HyDE vector $\mathbf{v}$ relate to the relevant documents. The script is `retrieval_eval.evaluation.hyde_geometry`; it makes no new LLM calls. Table 1 summarizes it.

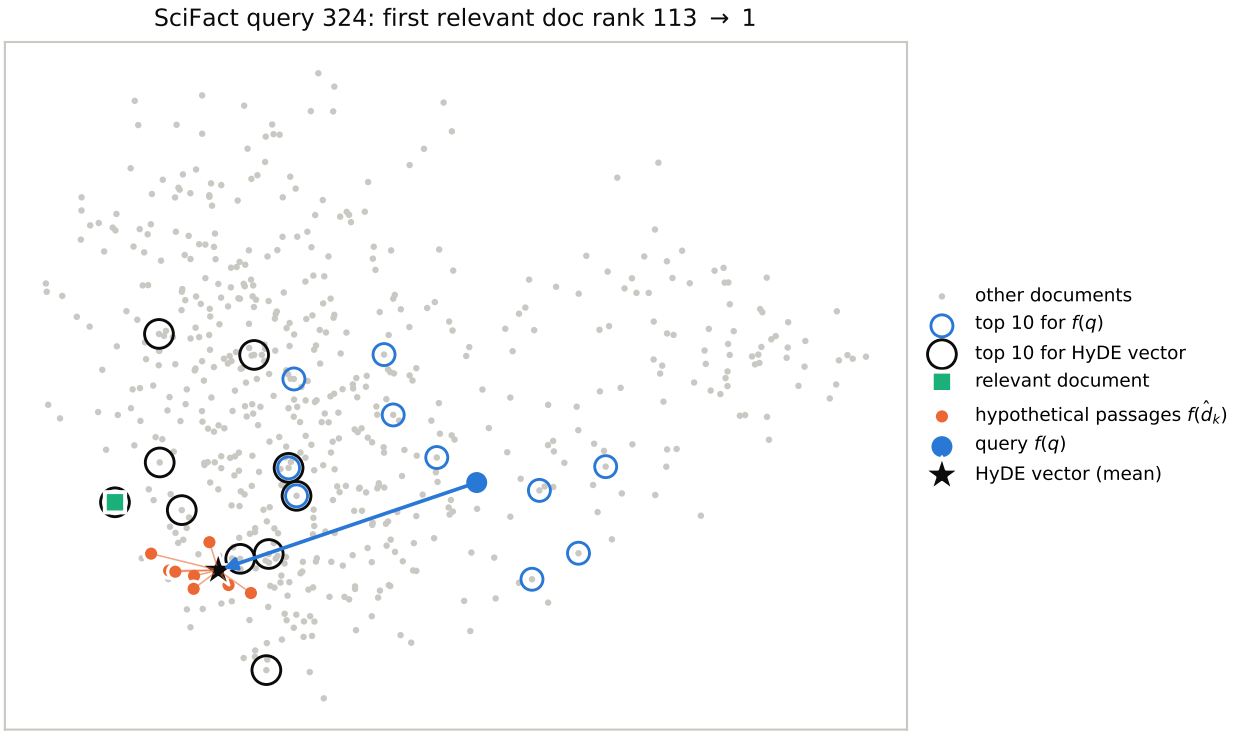



Figure 2: Real bge vectors for SciFact claim 324, projected to 2-D with PCA (fit on the plotted points: 300 nearest documents to $\mathbf{v}$, the query's top 10 and 300 random documents). The blue arrow is the move from $f(q)$ to the HyDE mean $\mathbf{v}$; thin orange lines join each passage to the mean. A 2-D projection distorts distances, so the rings (true top 10 in 384 dimensions) are the ground truth. Chosen as the query with the largest rank gain, so it is an illustration, not a typical case.

**Worked example.** SciFact (scientific claims, Section 5) claim 324 is "Deleting Raptor reduces G-CSF levels." Its relevant abstract never uses the word *Raptor*; it is about "mTOR signalling" regulating "G-CSF" in mammary tumours. Searching with $f(q)$ ranks it 113th, and none of the query's top 10 is relevant. The first gpt-oss passage begins "Raptor, a core component of the mechanistic target of rapamycin complex 1 (mTORC1)… granulocyte-colony stimulating factor (G-CSF)…": it is partly invented (it describes a mouse study that does not exist), but it supplies the missing link $\textit{Raptor} \rightarrow \textit{mTOR}$ in the vocabulary of an abstract. All eight passages do this in slightly different words. Their mean moves the search vector into the right neighbourhood: cosine to the relevant abstract rises from 0.646 to 0.836 and it is ranked **1st**. Figure 2 projects this query's vectors onto their first two principal components: the query's top-10 neighbours (blue rings) sit in the wrong area, while the eight passages cluster next to the relevant document and the top 10 around their mean (black rings) includes it.

**What the numbers show (bge).**

- **The passages land in document space.** Each passage is closer to the relevant documents than the query is (mean cosine 0.869 vs. 0.773 on SciFact, 0.718 vs. 0.606 on NFCorpus), and the mean $\mathbf{v}$ is closer still (0.879, 0.730): averaging removes per-sample noise.
- **The eight passages are nearly the same vector.** Their pairwise cosine is 0.96 on every dataset, so the 8 samples mostly add robustness. The diversity the paper hopes for is small

| | | mean cos to relevant docs | | | | median rank of 1st rel. | | % queries | passage |
|---|---|---|---|---|---|---|---|---|---|
| Encoder | Dataset | $f(q)$ | $f(\hat{d})$ | HyDE | $\cos(f(q), f(\hat{d}))$ | $f(q)$ | HyDE | rank better | words |
| bge | scifact | 0.773 | 0.869 | 0.879 | 0.864 | 1 | 1 | 29 | 407 |
| bge | nfcorpus | 0.606 | 0.718 | 0.730 | 0.810 | 2 | 2 | 31 | 417 |
| bge | fiqa | 0.731 | 0.798 | 0.814 | 0.856 | 2 | 2 | 34 | 502 |
| bge | trec-covid | 0.728 | 0.787 | 0.805 | 0.847 | 1 | 1 | 10 | 408 |
| contriever | scifact | 0.513 | 0.500 | 0.569 | 0.505 | 1 | 2 | 23 | 407 |
| contriever | nfcorpus | 0.296 | 0.327 | 0.373 | 0.404 | 2 | 6 | 25 | 417 |
| contriever | fiqa | 0.444 | 0.344 | 0.397 | 0.422 | 11 | 430 | 16 | 502 |
| contriever | trec-covid | 0.468 | 0.411 | 0.476 | 0.392 | 2 | 1 | 46 | 408 |

Table 1: HyDE geometry, computed from the cached generations. Cosines use unit vectors; ranks use the scoring of the actual run. "% rank better" is the share of queries whose first relevant document moves up when $f(q)$ is replaced by the HyDE vector.

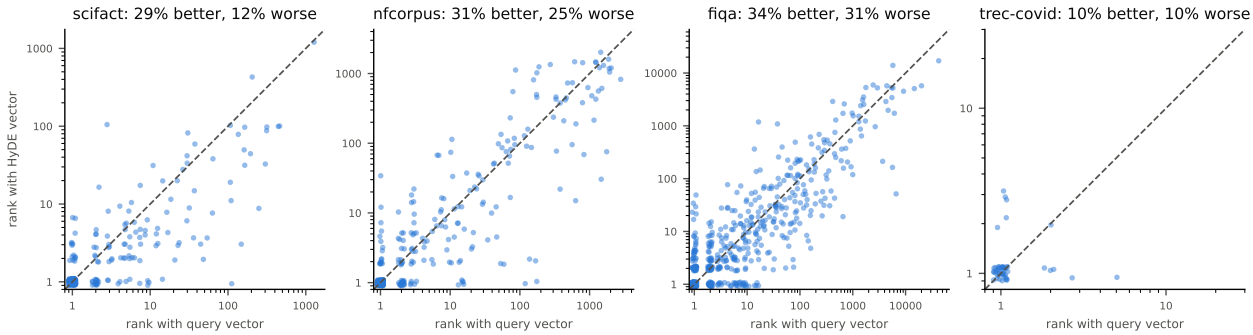



Figure 3: Rank of the first relevant document with the query vector ($x$) vs. with the HyDE vector ($y$), bge, one dot per query (log scales, slight jitter). Points below the diagonal are queries HyDE helps.

with gpt-oss at temperature 0.7.

- **Closer to everything, not only to the right thing.** $\mathbf{v}$ is also closer to random documents (mean cosine 0.56 → 0.65 on SciFact), because document-like vectors share a common direction. What matters for search is the *order*, so Figure 3 compares ranks. The first relevant document moves up for 29 % of SciFact queries and down for 12 %; on NFCorpus and FiQA the shifts are larger in both directions (31/25 % and 34/31 %); on TREC-COVID the first relevant document is usually already at rank 1. This matches the nDCG@10 gains: large on SciFact (+0.056), small on FiQA (+0.007).

**The published setup does not reproduce with gpt-oss.** HyDE-Contriever reaches only 0.556 / 0.063 / 0.434 (SciFact / FiQA / TREC-COVID) against the published 0.691 / 0.273 / 0.593, which is *below* Contriever alone. Contriever itself reproduces exactly, so the cause is in the generated passages. gpt-oss-120b writes long answers (median 507 words on FiQA, many with markdown headings and tables), far longer than the paper's passages. Contriever's mean-pooled vectors are not normalized, so these long, formatted passages dominate the averaged query vector. Stripping markdown and cutting each passage to 100 words (HyDE-Contriever-100w) recovers SciFact (0.684) and FiQA (0.285, above the published 0.273) but hurts TREC-COVID (0.351), so it is not a general fix, and we report it as our variant, not a reproduction. With the normalized

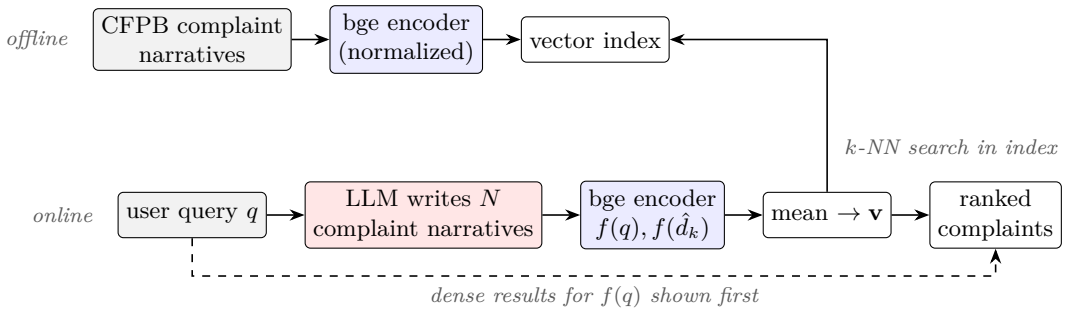
bge encoder the same passages work well: HyDE-bge improves on Dense on SciFact (+0.056), NFCorpus (+0.021) and FiQA (+0.007) and ties on TREC-COVID (−0.002). HyDE is therefore sensitive to the generator's output style in a way the paper, using one generator, did not have to address.

**Why Contriever fails.** The same passages hurt Contriever, and the norms explain it. Contriever's query vectors have length ≈ 1.35, the passages ≈ 2.0 and corpus documents ≈ 1.75. In the unnormalized mean the eight long passages dominate: $\mathbf{v}$ keeps only a cosine of 0.51–0.62 with $f(q)$, so the question itself is largely lost. On FiQA, where gpt-oss passages are longest (502 words) and often formatted as markdown tables, the passages are *farther* from the relevant documents than the query is (0.344 vs. 0.444), and the median rank of the first relevant document falls from 11 to 430. Cutting passages to 100 words reduces their norm to ≈ 1.65, which is why HyDE-Contriever-100w recovers FiQA. HyDE thus works when (i) the generator writes passages in the corpus's style and (ii) the encoder gives each vector an equal say. With bge both hold, and HyDE-bge is one of the best systems we tested.

# 4 Applying HyDE to CCDB Search

We chose HyDE for CCDB Search for three reasons from this study. (i) It was the strongest single first-stage retriever on three of the four test collections: HyDE-bge has the highest nDCG@10 of any non-reranking system on SciFact, NFCorpus and FiQA, and beats BM25 on all four (Part II). (ii) It needs no training and no labelled data, which matters for a collection with no relevance judgments. (iii) It reuses an ordinary dense index: the LLM only rewrites the query, so the narratives are embedded once and never sent to the LLM.

**Planned pipeline.** The narratives are embedded once with a normalized encoder and stored in a vector index. At query time the LLM writes a few complaint-style narratives for the query, these and the query are embedded and averaged, and the index returns the nearest complaints. Because the LLM step takes seconds, plain dense results for $f(q)$ can be shown first and replaced when the HyDE results arrive.




**Design choices.** The measurements of Section 3 give a recipe, and a list of things that break HyDE:

- **Use a normalized encoder** (bge-style, unit vectors). With Contriever's unnormalized vectors the same passages made results *worse* than no HyDE at all.
- **Keep the query in the mean.** $f(q)$ anchors the vector to what was asked; when it was drowned out (Contriever) relevant documents fell from rank 11 to rank 430 on FiQA.

- **Make the prompt describe complaint narratives.** HyDE works because the passages look like corpus documents. The paper's prompts name the document type ("scientific paper passage", "financial article"); for the CCDB the prompt should ask for a consumer complaint narrative, e.g. "Please write a consumer complaint to the CFPB describing this problem", in plain first-person prose of typical narrative length. gpt-oss's default style here (400–500-word markdown with headings and tables) is far from how consumers write, and on Contriever that mismatch cost most of the gain.
- **FiQA is the closest test, and the gain there was small.** FiQA (financial questions answered from forum posts) is the nearest of our collections to the CCDB. HyDE-bge improved on Dense there by only +0.007 nDCG@10, and 31 % of queries got worse. The financial domain alone does not guarantee a large benefit; matching the narrative style (previous point) is likely what matters.
- **Try fewer samples.** The eight passages were nearly the same vector (cosine 0.96), so 2–4 samples may give most of the benefit at lower cost. This should be tested on CCDB queries.
- **Plan for latency.** HyDE took 10–19 s per query here (the slowest of 8 parallel LLM calls), against about 10 ms for Dense. For an interactive search this needs a faster or smaller generator, streaming, caching of frequent queries, or a fallback that shows Dense results first.
- **Expect some queries to get worse.** 10–31 % of queries ranked their first relevant document lower with HyDE. Fusing HyDE with BM25 or Dense (as Hybrid does) is a cheap hedge.
- **Evaluate on the CCDB itself.** Gains ranged from +0.056 (SciFact) to −0.002 (TREC-COVID), so a small set of judged CCDB queries (e.g. labelled with the LLM judge of Section 9.1, after checking it against a few human labels) is needed before relying on it.

# Part II
# Comparison with other semantic search methods

## 5 Test collections

The CFPB Consumer Complaint Database has no relevance judgments, so the comparison uses four public BEIR [18] test collections where, as in complaint search, the query and the relevant text tend to be worded differently (Table 2). FiQA, natural-language finance questions answered from forum posts, is the closest to the CCDB.

Table 2: Collections (BEIR test splits). All queries have at least one relevant document.

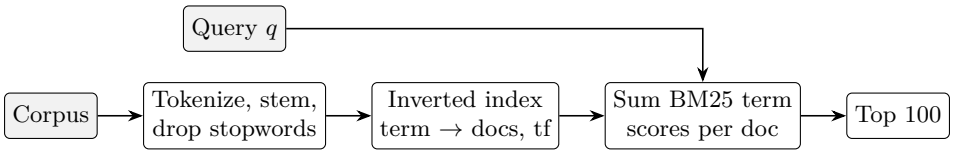
| Collection | Docs | Queries | Qrels | Grades | What it tests |
|---|---|---|---|---|---|
| SciFact [21] | 5,183 | 300 | 339 | 1 | Scientific claims → supporting abstracts |
| NFCorpus [3] | 3,633 | 323 | 12,334 | 1–2 | Lay health queries → medical papers |
| FiQA-2018 [13] | 57,638 | 648 | 1,706 | 1 | Natural-language finance questions → forum answers |
| TREC-COVID [20] | 171,332 | 50 | 66,336 | 0–2 | COVID questions → CORD-19 abstracts; deep pools |

# 6 The methods compared

This section defines each method HyDE is compared with, in a few sentences, and shows its data flow. $q$ is the query, $d$ a document, and every method ends with a ranked list of document ids.

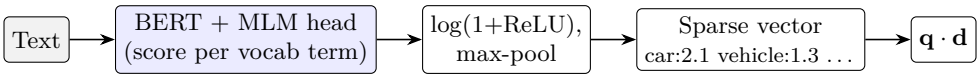
## 6.1 BM25 (lexical baseline) [9, 18]

BM25 scores a document by the query terms it contains. Each matching term contributes its inverse document frequency (rare terms count more), damped by term frequency (controlled by $k_1$) and normalized by document length (controlled by $b$): $\text{score}(q,d) = \sum_{t \in q} \text{idf}(t)\, \frac{tf_{t,d}(k_1+1)}{tf_{t,d}+k_1(1-b+b\,|d|/\overline{|d|})}$. Text is tokenized and stemmed first, so "running" matches "run", but "car" never matches "automobile". An inverted index makes this very fast. BM25-BEIR is the same formula with BEIR's parameters and with title and body scored separately and summed.




## 6.2 SPLADE (learned sparse retrieval) [5, 6]

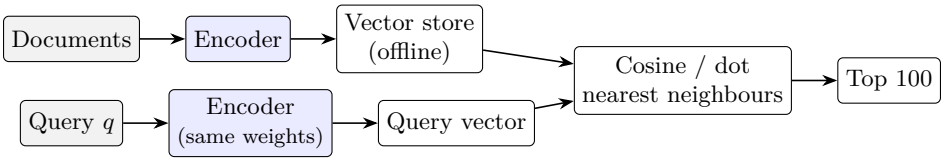
SPLADE keeps BM25's "bag of weighted terms" form but lets a BERT model choose the terms and weights. For every token position, the masked-language-model head predicts a score for each of the 30,522 vocabulary terms. These are passed through $\log(1 + \text{ReLU}(\cdot))$ and max-pooled over positions, giving one sparse vector per text. The vector also contains related terms that never appear in the text (*expansion*, e.g. "car" → "vehicle"). Relevance is the dot product of the query and document vectors, which an inverted index can compute.




*same model for queries and documents*

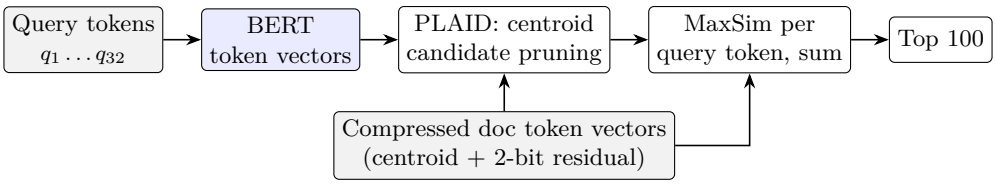
## 6.3 Dense bi-encoder (Dense = bge-small, Contriever) [8, 10, 23]

A transformer encodes the query and each document *separately* into a single dense vector (384 dimensions for bge-small, 768 for Contriever). Training places texts with the same meaning close together, so a query and a relevant document match even if they share no words. Relevance is cosine similarity (bge, normalized vectors) or a dot product (Contriever). Document vectors are computed once, offline; at query time only the query is encoded, followed by a nearest-neighbour search. bge-small is fine-tuned on large supervised relevance data; Contriever is trained without labels (contrastive learning on text crops), which explains its weaker results.



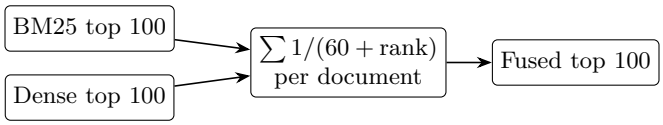
### 6.4 ColBERTv2 with PLAID (late interaction) [11, 14, 15]

ColBERT keeps one vector *per token* instead of one per text. A query is scored against a document by MaxSim: each query token finds its most similar document token, and these maxima are summed, score $= \sum_{i \in q} \max_{j \in d} \mathbf{q}_i \cdot \mathbf{d}_j$. This captures term-level matches with contextual meaning. Storing every token vector is expensive, so ColBERTv2 compresses each one to a cluster centroid plus a 2-bit residual. PLAID first uses the centroids to pick a small candidate set cheaply, then computes exact MaxSim only for those candidates.



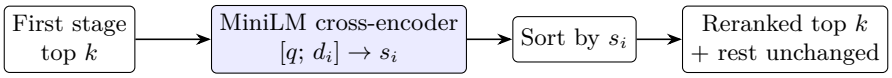

### 6.5 Hybrid: Reciprocal Rank Fusion [4]

RRF merges several ranked lists using only the rank positions, so scores on different scales (BM25 points vs. cosine similarities) never need calibrating. Each document receives $\sum_\ell 1/(60 + \text{rank}_\ell(d))$ over the lists that contain it. A document ranked highly by either list, or moderately by both, rises to the top. Here the lists are the BM25 and Dense top 100.




### 6.6 Cross-encoder reranking (Rerank, BM25+CE) [18, 22]

A cross-encoder reads the query and one document *together* in a single transformer pass ("[CLS] query [SEP] document"), so every query word can attend to every document word. It outputs one relevance score. This is more accurate than comparing separately computed vectors, but it must run once per (query, document) pair at query time, so it is applied only to a shortlist from a first-stage retriever: the Hybrid top 50 (Rerank) or the BM25-BEIR top 100 (BM25+CE).

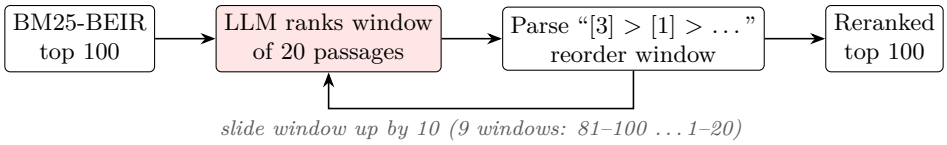



### 6.7 HyDE*: Hypothetical Document Embeddings [7]

HyDE is the method of Part I and is explained in detail in Section 2. In short, an LLM writes eight hypothetical answer passages, and the mean of their bge embeddings and the query's is searched exactly like Dense.

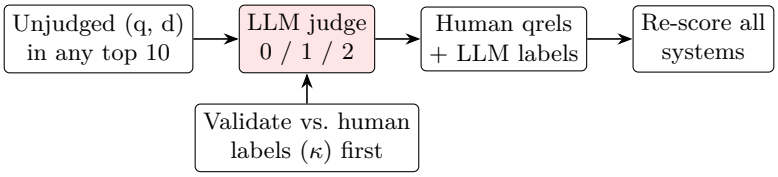
### 6.8 RankGPT: listwise LLM reranking [17]

RankGPT gives an LLM the query and 20 numbered passages and asks it to output a ranking such as "[3] > [1] > [7] > ...". Because 100 passages do not fit well in one prompt, it uses a sliding window: rerank positions 81–100, slide up by 10 to rerank 71–90, and so on to the top. Strong documents can thus "bubble up" from the bottom of the list. That makes 9 LLM calls per query, which must run in sequence. Unparseable or missing ids are appended in their original order.




### 6.9 LLM relevance judge (evaluation aid, not a retriever)

Human relevance labels exist only for documents that were in the original judgment pool, and unjudged documents count as non-relevant. Following Thomas et al. [19], an LLM (Qwen3.5) is shown the query and one document and asked for a 0–2 relevance grade, after considering intent match (M) and trustworthiness (T). We first measure its agreement with human labels, then use it only to label the unjudged documents in each system's top 10. Human labels are never overwritten.




## 7 Systems and evaluation

There is one script per method; all see the same corpus (title + body) and the test queries, and each returns a top-100 list. Nothing was tuned on the test queries. Section 6 explains how each method works; Table 3 lists the exact configurations; "repro." marks a system configured to match a published result.

Table 3: Systems. LLM calls go through the Databricks Unity Gateway; every call is cached on disk. * The method planned for CCDB Search over the CFPB Consumer Complaint Database (Section 4).

| System | Source | Configuration |
|---|---|---|
| BM25 (baseline) | BEIR [18], [9] | `bm25s`, Lucene variant, $k_1$=1.2, $b$=0.75, stopwords, Snowball stemmer |
| BM25-BEIR (repro.) | BEIR [18] | Anserini setting: $k_1$=0.9, $b$=0.4, Porter stemmer, title and body indexed as separate fields with scores summed |
| SPLADE (repro.) | [5, 6] | `naver/splade_v2_distil`, 256 tokens, sparse dot product |
| Dense (repro.) | DPR [10]; [2, 23] | `bge-small-en-v1.5`, CLS pooling, normalized, query instruction prefix, exact cosine search |
| Contriever (repro.) | [8], HyDE T2 [7] | `facebook/contriever` (unsupervised), mean pooling, unnormalized dot product |
| ColBERTv2 (repro.) | [11, 14, 15] | `colbertv2.0` via PyLate, PLAID index, 2-bit residuals, doc 300 / query 32 tokens |
| Hybrid | Cormack et al. [4] | RRF of BM25 and Dense, $\sum 1/(60 + r)$ with $r$ from 1 as in the paper |
| Rerank | BEIR [18], [22] | `ms-marco-MiniLM-L-6-v2` rescores the Hybrid top 50 |
| BM25+CE (repro.) | BEIR T2 [18] | Same cross-encoder on the BM25-BEIR top 100 |
| HyDE-bge* | HyDE [7] | gpt-oss-120b writes 8 passages per query (temp. 0.7, 512 tokens, the paper's prompts); query vector = mean of query and passage embeddings; bge encoder |
| HyDE-Contriever (repro.) | HyDE [7] | As above with the paper's Contriever encoder |
| HyDE-Contriever-100w | ours | As above, but passages are stripped of markdown and cut to 100 words before encoding (Section 3) |
| RankGPT (repro.) | Sun et al. [17] | Official prompts, passages cut to 300 words, sliding window 20 / step 10 over the BM25-BEIR top 100; gpt-oss-120b, temperature 0 |

Implementation notes from checking each paper:

- BEIR's BM25 baseline is Anserini/Lucene with $k_1$=0.9, $b$=0.4 and multi-field indexing. Our default BM25 keeps the common $k_1$=1.2, $b$=0.75; it scores slightly higher on SciFact and FiQA and lower on TREC-COVID, so we report both.
- PyTerrier's `RRFusion` [1, 12] uses 0-based ranks, i.e. $1/(k + r - 1)$ with 1-based $r$; we follow the original paper.
- DPR used an unnormalized dot product, FAISS HNSW search and a linear BM25 hybrid ($\lambda$=1.1). We use cosine scoring as specified by the bge model card, exact search, and RRF.
- SPLADE scores 0.703 on SciFact at 512 tokens; the paper's 256-token setting gives 0.693, matching the published number exactly.
- The LLMs differ from the papers': gpt-oss-120b (reasoning effort "low") replaces text-davinci-003 (HyDE) and gpt-3.5/gpt-4 (RankGPT). The judge is Qwen3.5-122B-A10B with thinking off, deliberately a different model from the one doing the reranking.

**Evaluation.** `ir-measures` (pytrec_eval) on the graded test qrels: nDCG@10 (primary), Recall@100 (secondary), P@10 and AP@100; unjudged documents count as non-relevant, as in BEIR. $R_{cap}$@100 divides by $\min(|\text{rel}|, 100)$, as in BEIR Table 9. Each system is compared with BM25 on per-query nDCG@10 with a paired $t$-test. BEIR runs no significance tests; the test is our addition, following Smucker et al. [16]. Latency is the mean wall-clock time per query on an Apple M5 Max (PyTorch MPS), model load excluded. Seeds are fixed at 0.

# 8 Reproductions against published results

Table 6 puts each reproduction next to the number its paper reports. Six of seven methods reproduce: the largest gap is 0.013 (BM25-BEIR on TREC-COVID), and SPLADE, bge-small and Contriever agree to within 0.003 on every collection. BM25-BEIR also matches BEIR's capped Recall@100 (TREC-COVID 0.499 vs. 0.498), and the cross-encoder setup matches BEIR's BM25+CE rows to within 0.005. These agreements matter because they show the pipeline, data and metric code are right, so the differences in Section 9 come from the methods.

## 8.1 RankGPT with gpt-oss beats the published gpt-3.5 results

Reranking the same BM25 top 100, RankGPT with gpt-oss-120b scores 0.765 / 0.383 / 0.878 (SciFact / NFCorpus / TREC-COVID), against 0.704 / 0.356 / 0.767 for gpt-3.5-turbo and 0.750 / 0.385 / 0.855 for gpt-4 in the paper. The gpt-4 rows reranked gpt-3.5's top 30, not BM25's top 100, so that comparison is not like-for-like. Our BM25-BEIR first stage is also stronger than the paper's Pyserini BM25 on TREC-COVID (0.669 vs. 0.595), which accounts for part of the gap. Output problems were rare: across 11,022 windows, 12 duplicate ids, 994 missing ids (appended in original order by the official `receive_permutation`) and 4 empty responses. HyDE is the exception: with gpt-oss-120b and Contriever it does not reproduce the paper, for reasons explained in Section 3.

# 9 Results: semantic search vs. the BM25 baseline

Table 4: nDCG@10 per system (rows) and collection (columns). $\Delta$ = difference from BM25; $^*$ paired $t$-test $p < 0.05$, $^{**}$ $p < 0.001$. Best per column in bold.

| | | scifact | | nfcorpus | | fiqa | | trec-covid | |
|---|---|---|---|---|---|---|---|---|---|
| System | Family | nDCG | $\Delta$ | nDCG | $\Delta$ | nDCG | $\Delta$ | nDCG | $\Delta$ |
| BM25 | Lexical | 0.679 | – | 0.318 | – | 0.253 | – | 0.609 | – |
| BM25-BEIR | Lexical | 0.656 | -0.023$^*$ | 0.320 | +0.002 | 0.236 | -0.017$^*$ | 0.669 | +0.060$^*$ |
| SPLADE | Learned sparse | 0.693 | +0.013 | 0.334 | +0.016 | 0.336 | +0.083$^{**}$ | 0.710 | +0.101$^*$ |
| Dense | Dense | 0.713 | +0.034$^*$ | 0.343 | +0.025$^*$ | 0.403 | +0.150$^{**}$ | 0.757 | +0.147$^{**}$ |
| Contriever | Dense | 0.649 | -0.030 | 0.317 | -0.001 | 0.245 | -0.008 | 0.274 | -0.335$^{**}$ |
| ColBERTv2 | Late interaction | 0.691 | +0.012 | 0.339 | +0.021$^*$ | 0.355 | +0.102$^{**}$ | 0.731 | +0.121$^*$ |
| HyDE-bge | LLM | **0.769** | +0.090$^{**}$ | 0.364 | +0.046$^{**}$ | 0.410 | +0.157$^{**}$ | 0.755 | +0.146$^{**}$ |
| HyDE-Contriever | LLM | 0.556 | -0.123$^{**}$ | 0.191 | -0.128$^{**}$ | 0.063 | -0.190$^{**}$ | 0.434 | -0.175$^{**}$ |
| HyDE-Contriever-100w | LLM | 0.684 | +0.005 | 0.289 | -0.030$^*$ | 0.285 | +0.032$^*$ | 0.351 | -0.259$^{**}$ |
| Hybrid | Hybrid | 0.724 | +0.045$^{**}$ | 0.361 | +0.042$^{**}$ | 0.366 | +0.113$^{**}$ | 0.778 | +0.169$^{**}$ |
| Rerank | Reranker | 0.697 | +0.017 | 0.357 | +0.038$^{**}$ | 0.371 | +0.118$^{**}$ | 0.779 | +0.170$^{**}$ |
| BM25+CE | Reranker | 0.689 | +0.010 | 0.350 | +0.032$^{**}$ | 0.351 | +0.098$^{**}$ | 0.762 | +0.152$^{**}$ |
| RankGPT | LLM | 0.765 | +0.086$^{**}$ | **0.383** | +0.064$^{**}$ | **0.429** | +0.176$^{**}$ | **0.878** | +0.268$^{**}$ |

Table 5: Secondary metrics. $R_{cap}$@100 divides by $\min(|rel|, 100)$ as in BEIR Table 9; J@10 is a diagnostic (share of top-10 docs with any human judgment).

| | scifact | | | | | nfcorpus | | | | | fiqa | | | | | trec-covid | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| System | R@100 | $R_{cap}$ | P@10 | AP | J@10 | R@100 | $R_{cap}$ | P@10 | AP | J@10 | R@100 | $R_{cap}$ | P@10 | AP | J@10 | R@100 | $R_{cap}$ | P@10 | AP | J@10 |
| BM25 | 0.913 | 0.913 | 0.089 | 0.638 | 0.089 | 0.247 | 0.260 | 0.229 | 0.143 | 0.229 | 0.553 | 0.553 | 0.070 | 0.205 | 0.070 | 0.110 | 0.472 | 0.642 | 0.074 | 0.876 |
| BM25-BEIR | 0.905 | 0.905 | 0.088 | 0.612 | 0.088 | 0.250 | 0.263 | 0.231 | 0.143 | 0.231 | 0.542 | 0.542 | 0.064 | 0.193 | 0.064 | 0.114 | 0.499 | 0.724 | 0.081 | 0.906 |
| SPLADE | 0.920 | 0.920 | 0.090 | 0.661 | 0.090 | 0.277 | 0.293 | 0.240 | 0.157 | 0.240 | 0.621 | 0.621 | 0.090 | 0.280 | 0.090 | 0.123 | 0.550 | 0.764 | 0.088 | 0.940 |
| Dense | 0.942 | 0.942 | 0.094 | 0.674 | 0.094 | 0.311 | 0.326 | 0.257 | 0.156 | 0.257 | 0.696 | 0.696 | 0.111 | 0.346 | 0.111 | 0.133 | 0.561 | 0.794 | 0.099 | 0.878 |
| Contriever | 0.926 | 0.926 | 0.089 | 0.606 | 0.089 | 0.294 | 0.308 | 0.231 | 0.148 | 0.231 | 0.562 | 0.562 | 0.069 | 0.199 | 0.069 | 0.037 | 0.172 | 0.270 | 0.016 | 0.346 |
| ColBERTv2 | 0.922 | 0.922 | 0.090 | 0.657 | 0.090 | 0.288 | 0.303 | 0.238 | 0.159 | 0.238 | 0.638 | 0.638 | 0.094 | 0.299 | 0.094 | 0.128 | 0.555 | 0.794 | 0.094 | 0.952 |
| HyDE-bge | 0.983 | 0.983 | 0.102 | 0.727 | 0.102 | 0.333 | 0.347 | 0.275 | 0.166 | 0.275 | 0.727 | 0.727 | 0.112 | 0.352 | 0.112 | 0.139 | 0.584 | 0.796 | 0.109 | 0.876 |
| HyDE-Contriever | 0.886 | 0.886 | 0.079 | 0.508 | 0.079 | 0.212 | 0.220 | 0.145 | 0.072 | 0.145 | 0.187 | 0.187 | 0.016 | 0.050 | 0.016 | 0.064 | 0.296 | 0.466 | 0.034 | 0.546 |
| HyDE-Contriever-100w | 0.957 | 0.957 | 0.093 | 0.634 | 0.093 | 0.297 | 0.308 | 0.213 | 0.124 | 0.213 | 0.649 | 0.649 | 0.083 | 0.231 | 0.083 | 0.051 | 0.241 | 0.374 | 0.025 | 0.442 |
| Hybrid | 0.968 | 0.968 | 0.096 | 0.681 | 0.096 | 0.312 | 0.328 | 0.263 | 0.174 | 0.263 | 0.693 | 0.693 | 0.104 | 0.305 | 0.104 | 0.133 | 0.554 | 0.808 | 0.100 | 0.934 |
| Rerank | 0.968 | 0.968 | 0.093 | 0.656 | 0.093 | 0.312 | 0.328 | 0.251 | 0.173 | 0.251 | 0.693 | 0.693 | 0.104 | 0.310 | 0.104 | 0.133 | 0.554 | 0.858 | 0.102 | 0.974 |
| BM25+CE | 0.905 | 0.905 | 0.091 | 0.649 | 0.091 | 0.250 | 0.263 | 0.244 | 0.161 | 0.244 | 0.542 | 0.542 | 0.094 | 0.287 | 0.094 | 0.114 | 0.499 | 0.824 | 0.089 | 0.962 |
| RankGPT | 0.905 | 0.905 | 0.098 | 0.731 | 0.098 | 0.250 | 0.263 | 0.273 | 0.172 | 0.273 | 0.542 | 0.542 | 0.108 | 0.360 | 0.108 | 0.114 | 0.499 | 0.924 | 0.095 | 0.976 |

Table 6: Our runs vs. published results (nDCG@10 unless noted). "Ours" is the matching system from this study; $\Delta$ = ours − published. Rows without a match are reference points only.

| | | scifact | | nfcorpus | | fiqa | | trec-covid | |
|---|---|---|---|---|---|---|---|---|---|
| Published (source) | Ours | pub. | $\Delta$ | pub. | $\Delta$ | pub. | $\Delta$ | pub. | $\Delta$ |
| BM25, Anserini k1=0.9 b=0.4 (BEIR T2) [18] | BM25-BEIR | 0.665 | -0.009 | 0.325 | -0.005 | 0.236 | +0.000 | 0.656 | +0.013 |
| BM25 top-100 + MiniLM-L6 CE (BEIR T2) [18] | BM25+CE | 0.688 | +0.001 | 0.350 | +0.000 | 0.347 | +0.004 | 0.757 | +0.005 |
| bge-small-en-v1.5 (model card, MTEB) [2] | Dense | 0.713 | -0.000 | 0.343 | +0.000 | 0.403 | +0.000 | 0.759 | -0.002 |
| DistilSPLADE-max (SPLADE v2 T2) [5] | SPLADE | 0.693 | -0.000 | 0.334 | +0.000 | 0.336 | -0.000 | 0.710 | +0.000 |
| ColBERTv2 (ColBERTv2 T5a) [15] | ColBERTv2 | 0.693 | -0.002 | 0.338 | +0.001 | 0.356 | -0.001 | 0.738 | -0.007 |
| Contriever, unsupervised (HyDE T2) [7] | Contriever | 0.649 | +0.000 | – | | 0.245 | -0.000 | 0.273 | +0.001 |
| HyDE, text-davinci-003 + Contriever (HyDE T2) [7] | HyDE-Contriever | 0.691 | -0.135 | – | | 0.273 | -0.210 | 0.593 | -0.159 |
| RankGPT, gpt-3.5-turbo on BM25 top-100 (RankGPT T1) [17] | RankGPT | 0.704 | +0.060 | 0.356 | +0.027 | – | | 0.767 | +0.111 |
| BM25, Pyserini (RankGPT T1) [17] | – | 0.679 | | 0.307 | | – | | 0.595 | |
| RankGPT, gpt-4 on gpt-3.5 top-30 (RankGPT T1) [17] | – | 0.750 | | 0.385 | | – | | 0.855 | |
| monoT5 220M on BM25 top-100 (RankGPT T1) [17] | – | 0.734 | | 0.374 | | – | | 0.783 | |
| DPR, NQ-trained (BEIR T2) [18] | – | 0.318 | | 0.189 | | 0.112 | | 0.332 | |
| ColBERT v1 (BEIR T2) [18] | – | 0.671 | | 0.305 | | 0.317 | | 0.677 | |
| *Recall@100* | | | | | | | | | |
| BM25 (BEIR T9) [18] | BM25-BEIR | 0.908 | -0.003 | 0.250 | +0.013 | 0.539 | +0.003 | 0.498 | +0.001 |
| bge-small-en-v1.5 (model card, MTEB; uncapped) [2] | Dense | 0.942 | -0.000 | 0.311 | -0.000 | 0.696 | +0.001 | 0.133 | +0.000 |

Table 7: nDCG@10 with LLM labels filling the judgment holes (Thomas et al. [19] prompt, Qwen3.5-122B judge). $\Delta$ vs. BM25 under the same augmented qrels; * $p < 0.05$, ** $p < 0.001$.

| | scifact | | | nfcorpus | | | fiqa | | | trec-covid | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| System | orig. | aug. | $\Delta$ | orig. | aug. | $\Delta$ | orig. | aug. | $\Delta$ | orig. | aug. | $\Delta$ |
| BM25 | 0.679 | 0.705 | – | 0.318 | 0.508 | – | 0.253 | 0.506 | – | 0.609 | 0.624 | – |
| BM25-BEIR | 0.656 | 0.661 | -0.045** | 0.320 | 0.491 | -0.017* | 0.236 | 0.537 | +0.031** | 0.669 | 0.675 | +0.051* |
| SPLADE | 0.693 | 0.687 | -0.018 | 0.334 | 0.543 | +0.035** | 0.336 | 0.603 | +0.096** | 0.710 | 0.719 | +0.095* |
| Dense | 0.713 | 0.712 | +0.007 | 0.343 | 0.537 | +0.029* | 0.403 | 0.636 | +0.130** | 0.757 | 0.793 | +0.169** |
| Contriever | 0.649 | 0.671 | -0.034* | 0.317 | 0.500 | -0.008 | 0.245 | 0.510 | +0.004 | 0.274 | 0.329 | -0.295** |
| ColBERTv2 | 0.691 | 0.687 | -0.018 | 0.339 | 0.538 | +0.030* | 0.355 | 0.622 | +0.116** | 0.731 | 0.741 | +0.117* |
| HyDE-bge | 0.769 | 0.793 | +0.087** | 0.364 | 0.547 | +0.039* | 0.410 | 0.751 | +0.245** | 0.755 | 0.829 | +0.205** |
| HyDE-Contriever | 0.556 | 0.592 | -0.113** | 0.191 | 0.315 | -0.193** | 0.063 | 0.199 | -0.307** | 0.434 | 0.636 | +0.012 |
| HyDE-Contriever-100w | 0.684 | 0.721 | +0.016 | 0.289 | 0.440 | -0.068** | 0.285 | 0.652 | +0.146** | 0.351 | 0.504 | -0.120* |
| Hybrid | 0.724 | 0.743 | +0.038** | 0.361 | 0.573 | +0.065** | 0.366 | 0.639 | +0.132** | 0.778 | 0.793 | +0.169** |
| Rerank | 0.697 | 0.711 | +0.006 | 0.357 | 0.566 | +0.058** | 0.371 | 0.651 | +0.145** | 0.779 | 0.790 | +0.166** |
| BM25+CE | 0.689 | 0.696 | -0.010 | 0.350 | 0.552 | +0.044** | 0.351 | 0.622 | +0.116** | 0.762 | 0.766 | +0.142** |
| RankGPT | 0.765 | 0.818 | +0.113** | 0.383 | 0.619 | +0.111** | 0.429 | 0.799 | +0.293** | 0.878 | 0.892 | +0.268** |

Table 8: Mean end-to-end latency per query (ms; model load excluded; LLM methods include the gateway call time) and one-time index/encode cost (s).

| | scifact | | nfcorpus | | fiqa | | trec-covid | |
|---|---|---|---|---|---|---|---|---|
| System | ms/q | setup s | ms/q | setup s | ms/q | setup s | ms/q | setup s |
| BM25 | 0.2 | 0 | 0.1 | 0 | 0.5 | 2 | 1.3 | 7 |
| BM25-BEIR | 0.2 | 0 | 0.1 | 0 | 0.4 | 2 | 1.7 | 8 |
| SPLADE | 10.6 | 20 | 8.3 | 14 | 34.3 | 168 | 83.2 | 588 |
| Dense | 5.4 | 14 | 5.4 | 10 | 6.9 | 92 | 9.1 | 380 |
| Contriever | 6.6 | 40 | 6.2 | 31 | 8.6 | 276 | 11.6 | 1,037 |
| ColBERTv2 | 255.0 | 113 | 162.2 | 48 | 259.8 | 1,006 | 383.6 | 5,397 |
| HyDE-bge | 12,338.5 | 14 | 19,194.9 | 10 | 13,451.1 | 93 | 9,555.9 | 354 |
| HyDE-Contriever | 12,383.6 | 40 | 19,239.6 | 31 | 13,495.7 | 276 | 9,601.6 | 1,037 |
| HyDE-Contriever-100w | 12,341.9 | 40 | 19,197.5 | 31 | 13,450.0 | 276 | 9,560.0 | 1,037 |
| Hybrid | 5.6 | 14 | 5.5 | 10 | 7.4 | 94 | 10.4 | 387 |
| Rerank | 86.8 | 14 | 91.3 | 10 | 93.6 | 94 | 94.2 | 387 |
| BM25+CE | 203.2 | 0 | 188.5 | 0 | 151.4 | 2 | 252.3 | 8 |
| RankGPT | 43,859.7 | 0 | 35,855.8 | 0 | 49,051.0 | 2 | 25,639.8 | 8 |

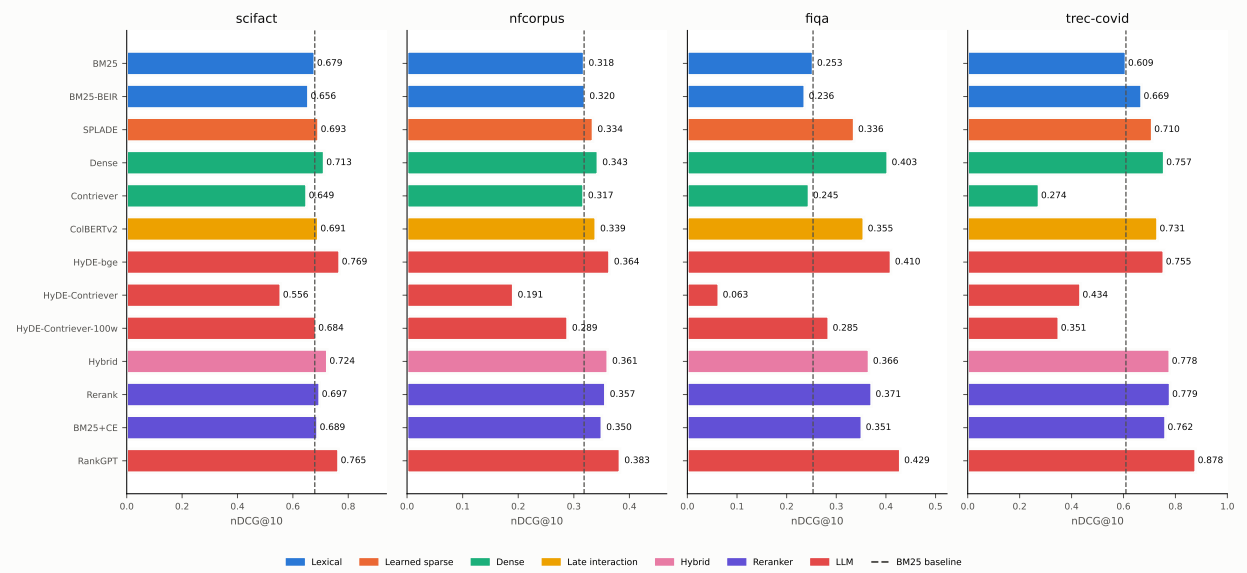



Figure 4: nDCG@10 per system and collection; dashed line = BM25 baseline.

**Natural-language questions gain most.** On FiQA every system built on a supervised encoder beats BM25 with $p < 0.001$: Dense +0.150 (313 wins, 86 losses), HyDE-bge +0.157, and RankGPT +0.176. On TREC-COVID, Dense, Hybrid, Rerank, HyDE-bge and RankGPT gain +0.15 to +0.27; RankGPT wins 43 of 50 queries. On SciFact (declarative claims sharing technical terms with abstracts) and NFCorpus (short topic titles), the gains are +0.01 to +0.09, and several systems are not significantly better than BM25.

**Supervised training transfer matters more than architecture.** Unsupervised Contriever never beats BM25 and collapses on TREC-COVID (0.274), as in the HyDE paper. SPLADE and ColBERTv2, both trained on MS MARCO, beat BM25 on FiQA and TREC-COVID but trail the

much smaller bge-small, whose training mixture is broader. The bge release does not document its English training data at the dataset level, so some overlap with these collections cannot be ruled out (Section 11).

**Hybrid is the cheap, reliable choice.** RRF of BM25 and Dense is significantly better than BM25 with $p < 0.001$ on all four collections and is within 0.10 of the best system everywhere, at about 10 ms per query. Its one weakness is FiQA, where equal-weight fusion with the weak BM25 list lowers nDCG@10 from Dense's 0.403 to 0.366.

**Rerankers: the small cross-encoder helps a little, the LLM a lot.** The MS MARCO MiniLM cross-encoder adds +0.01 to +0.15 over BM25 but at most +0.005 over Hybrid. RankGPT on the same BM25 list as BM25+CE is far better on every collection (+0.076 on SciFact, +0.078 on FiQA, +0.116 on TREC-COVID).

**Recall@100.** Semantic first stages miss fewer relevant documents (Table 5), e.g. FiQA 0.553 → 0.727 (HyDE-bge) and NFCorpus 0.247 → 0.333. The rerankers cannot raise Recall@100 above that of their first stage, which matters for retrieval-augmented generation, where the LLM only sees what the first stage retrieved.

### 9.1 Judgment holes: does pooling bias favour BM25?

Semantic systems retrieve many documents that were never judged; scoring them as non-relevant could understate those systems. Following Thomas et al. [19], we labelled every unjudged document in any system's top 10 with an LLM judge using the paper's prompt: 63,029 pairs in total, 18 of which failed and stay unjudged.

**Validation first.** On 200 stratified human-judged TREC-COVID pairs, the judge agrees with humans at Cohen's $\kappa = 0.53$ (binary and 3-class linear-weighted) and 62.5% exact agreement, which is moderate agreement. The other qrels contain only relevant documents, so $\kappa$ is undefined there; the judge labels 80% (SciFact) and 91% (FiQA) of known-relevant documents as relevant, but only 27% on NFCorpus, whose relevance labels come from citation links rather than direct reading. The NFCorpus augmented numbers should therefore not be relied on.

**Results.** Table 7 rescored all systems with human labels plus LLM labels for the holes. All scores rise, because the holes contain relevant documents, and the semantic and LLM systems gain the most: on FiQA, BM25 rises 0.253 → 0.506 while HyDE-bge rises 0.410 → 0.751 and RankGPT 0.429 → 0.799; on TREC-COVID, Dense's gain over BM25 grows from +0.147 to +0.169. So pooling bias does affect these comparisons, and if anything it understates the semantic methods. One caveat applies to the LLM methods: an LLM judge may favour documents another LLM picked, even a different model.

## 10 Cost

Table 8 gives the latencies. BM25 answers in 0.1–1.7 ms, Dense and Hybrid in 5–10 ms, SPLADE in 8–83 ms (sparse matrix product, no inverted index), and ColBERTv2/PLAID in 160–380 ms. The cross-encoder adds about 80 ms over the Hybrid top 50 and 150–250 ms over the top 100. The LLM methods are in another class: HyDE takes 10–19 s per query (the slowest of 8 parallel generations) and RankGPT 26–49 s (9 sequential windows). These were measured with 24 concurrent gateway requests, under which per-call latency rose about 1.6×, so they overstate single-user latency but

not the order of magnitude. One-time setup also differs: on TREC-COVID, BM25 indexes in 7 s, bge-small encodes the corpus in 6 min, and ColBERTv2 needs 15 min to encode plus 75 min to build a 2.5 GB PLAID index. The LLM steps used 10,568 (HyDE), 11,022 (RankGPT) and 63,765 (judge) gateway calls, roughly 110M input tokens in total.

## 11 Threats to validity

- **Training overlap.** bge-small's fine-tuning mixture is not documented per dataset, so overlap with FiQA's or NFCorpus' training splits cannot be excluded. The ColBERTv2, SPLADE and Contriever results do not have this caveat.
- **Different LLMs.** gpt-oss-120b is not the model the HyDE and RankGPT papers used, and it is a much newer and stronger model than gpt-3.5. The RankGPT comparison shows what the method does with a current open-weight model, not a replication of the paper's numbers.
- **Lexical bias in the pools.** BEIR (Table 4) found TREC-COVID's pool still lexically biased despite its depth. J@10 shows our top-10 lists are mostly judged there (0.88 for both Dense and BM25), and filling the holes did not reverse any conclusion, but it remains a caveat.
- **Judge quality.** $\kappa = 0.53$ is moderate, and the judge disagrees with the NFCorpus labels. The augmented results are supporting evidence, not a replacement for the human qrels.
- **Multiple comparisons.** We ran 48 tests against BM25. With Bonferroni ($0.05/48 \approx 0.001$), only results marked ** survive; all conclusions above rest on those.
- **Small query set.** TREC-COVID has only 50 queries, although its main effects have $p < 10^{-4}$.
- **Fixed configurations and one machine.** Each system ran one configuration, chosen before seeing results; latency was measured on one laptop and is indicative only.

## 12 Conclusion

**For CCDB Search.** HyDE with a normalized encoder is a strong, training-free way to bridge the gap between short queries and long consumer-written narratives: it was the best first-stage retriever on three of four collections, and the geometry analysis shows why it works and when it breaks. Its costs are latency (10–19 s per query with 8 samples) and per-query variance (10–31 % of queries rank their first relevant document lower). Both can be managed: fewer samples, a faster generator, caching, fusion with dense or BM25 results, and showing dense results first. The prompt should ask for complaint narratives in the consumers' own style, and the system should be checked on a small set of judged CCDB queries before it is relied on.

**Semantic search in general.** Against BM25, semantic search helped on every collection:

- **SciFact:** yes, modestly for cheap methods (Hybrid +0.045, $p < 0.001$), substantially for LLM methods (HyDE-bge +0.090, RankGPT +0.086).
- **NFCorpus:** yes, modestly (Hybrid +0.042, RankGPT +0.064, both $p < 0.001$).
- **FiQA:** yes, strongly (Dense +0.150, RankGPT +0.176, $p < 0.001$); fusion with the weak BM25 list gives back a quarter of Dense's gain.

- **TREC-COVID:** yes, strongly (Hybrid +0.169, RankGPT +0.268, $p < 0.001$).

Semantic search helps most when queries are natural-language questions worded differently from the documents, which is the CCDB Search setting. A small supervised bi-encoder fused with BM25 delivers most of the gain for about 10 ms per query, and LLM reranking (RankGPT) delivers the best quality measured here at roughly 3,000–5,000 times the latency. HyDE sits between them: better than any other first-stage retriever on most collections, for the price of one LLM call per query.

## References


[1] PyTerrier documentation: Rank fusion (RRFusion). `https://pyterrier.readthedocs.io/`. Accessed 2026-11-03.

[2] BAAI. BAAI/bge-small-en-v1.5 model card (mteb results in model-index). `https://huggingface.co/BAAI/bge-small-en-v1.5`, 2023. Accessed 2026-11-04.

[3] Vera Boteva, Demian Gholipour, Artem Sokolov, and Stefan Riezler. A full-text learning to rank dataset for medical information retrieval. In *Advances in Information Retrieval (ECIR)*, pages 716–722, 2016.

[4] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.

[5] Thibault Formal, Carlos Lassance, Benjamin Piwowarski, and Stéphane Clinchant. SPLADE v2: Sparse lexical and expansion model for information retrieval. *arXiv preprint arXiv:2109.10086*, 2021.

[6] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. SPLADE: Sparse lexical and expansion model for first stage ranking. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2288–2292, 2021.

[7] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1762–1777, 2023.

[8] Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning. *Transactions on Machine Learning Research*, 2022.

[9] Chris Kamphuis, Arjen P. de Vries, Leonid Boytsov, and Jimmy Lin. Which BM25 do you mean? a large-scale reproducibility study of scoring variants. In *Advances in Information Retrieval (ECIR)*, pages 28–34, 2020.

[10] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020.

[11] Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48, 2020.

[12] Craig Macdonald and Nicola Tonellotto. Declarative experimentation in information retrieval using PyTerrier. In *Proceedings of the 2020 ACM SIGIR International Conference on Theory of Information Retrieval (ICTIR)*, pages 161–168, 2020.

[13] Macedo Maia, Siegfried Handschuh, André Freitas, Brian Davis, Ross McDermott, Manel Zarrouk, and Alexandra Balahur. WWW'18 open challenge: Financial opinion mining and question answering. In *Companion Proceedings of The Web Conference 2018*, pages 1941–1942, 2018.

[14] Keshav Santhanam, Omar Khattab, Christopher Potts, and Matei Zaharia. PLAID: An efficient engine for late interaction retrieval. In *Proceedings of the 31st ACM International Conference on Information and Knowledge Management (CIKM)*, pages 1747–1756, 2022.

[15] Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. ColBERTv2: Effective and efficient retrieval via lightweight late interaction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pages 3715–3734, 2022.

[16] Mark D. Smucker, James Allan, and Ben Carterette. A comparison of statistical significance tests for information retrieval evaluation. In *Proceedings of the 16th ACM Conference on Information and Knowledge Management (CIKM)*, pages 623–632, 2007.

[17] Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. Is ChatGPT good at search? investigating large language models as re-ranking agents. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 14918–14937, 2023.

[18] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.

[19] Paul Thomas, Seth Spielman, Nick Craswell, and Bhaskar Mitra. Large language models can accurately predict searcher preferences. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1930–1940, 2024.

[20] Ellen Voorhees, Tasmeer Alam, Steven Bedrick, Dina Demner-Fushman, William R. Hersh, Kyle Lo, Kirk Roberts, Ian Soboroff, and Lucy Lu Wang. TREC-COVID: Constructing a pandemic information retrieval test collection. *ACM SIGIR Forum*, 54(1), 2020.

[21] David Wadden, Shanchuan Lin, Kyle Lo, Lucy Lu Wang, Madeleine van Zuylen, Arman Cohan, and Hannaneh Hajishirzi. Fact or fiction: Verifying scientific claims. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7534–7550, 2020.

[22] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[23] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-Pack: Packaged resources to advance general Chinese embedding. *arXiv preprint arXiv:2309.07597*, 2023.

# A Reproducibility

Run `python3 -m retrieval_eval.data.fetch_beir && .venv/bin/python run.py`. Each method is one module in `retrieval_eval/methods/` (e.g. `python -m retrieval_eval.methods.splade`); scoring, the LLM judge and the published reference numbers are in `retrieval_eval/evaluation/`. ColBERTv2 runs in a separate `.venv-colbert` (PyLate pins an older torch). The LLM steps need `databricks auth` access to the Unity Gateway; with the shipped `llm_cache/` they make no new calls (`-no-llm` skips them). Library versions, model revisions and probe output are in `RESULT.md`; judge labels are in `llm_labels/` and augmented qrels in `qrels_aug/`.